Robot CLI Command Tool

Robot CLI Command Tool

1. Course Content
    - Course Overview
    - Learning Objectives
2. Preparation
3. Introduction to the CLI Tool
4. CLI Command Robot Control Communication Architecture
5. Controlling the Robot via CLI
    - 5.1 CLI Command Summary
    - 5.2 Tools Supported by Different Camera Vehicle Models
    - 5.3 Command Format
6. Case Demonstration
7. Source Code Analysis

1. Course Content

Course Overview

- This lesson explains the use of the robot CLI (Command Line Interface) command tool. Through the command-line interface, users can directly control the robot chassis movement, visual recognition, and various other functions.
- The CLI command tool encapsulates all MCP Tool interfaces, allowing you to quickly operate the robot without relying on an AI model, which facilitates function testing and debugging.

Learning Objectives

1. Understand the functional positioning and application scenarios of the robot CLI command tool
2. Become familiar with CLI command functions, parameters, and usage formats

> **Important**
>
> This lesson uses the depth camera (hp60c) as an example model for demonstration. The MCP and CLI tools of vehicle models with different hardware versions may vary slightly.

2. Preparation

```
ros2 launch yahboomcar_bringup mcp_service.launch.py use_camera:=true
```

```
[mcp_service-10]
[mcp_service-10]
[mcp_service-10]
[mcp_service-10]
[mcp_service-10]
[mcp_service-10]
[mcp_service-10]
[mcp_service-10]
[mcp_service-10] [09/07/26 11:13:26] INFO     Starting MCP server                transport.py:273
[mcp_service-10]                              'ROS-MCP-Service' with transport
[mcp_service-10]                              'streamable-http' on
[mcp_service-10]                              http://0.0.0.0:8000/mcp
[mcp_service-10] INFO:     Started server process [353120]
[mcp_service-10] INFO:     Waiting for application startup.
[mcp_service-10] INFO:     Application startup complete.
[mcp_service-10] INFO:     Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
[mcp_service-10] INFO:     127.0.0.1:55500 - "POST /mcp HTTP/1.1" 200 OK
[mcp_service-10] INFO:     127.0.0.1:55512 - "POST /mcp HTTP/1.1" 202 Accepted
[mcp_service-10] INFO:     127.0.0.1:55500 - "POST /mcp HTTP/1.1" 200 OK
```

ROSMASTER

ROSMASTER M1-Pro MCP Server

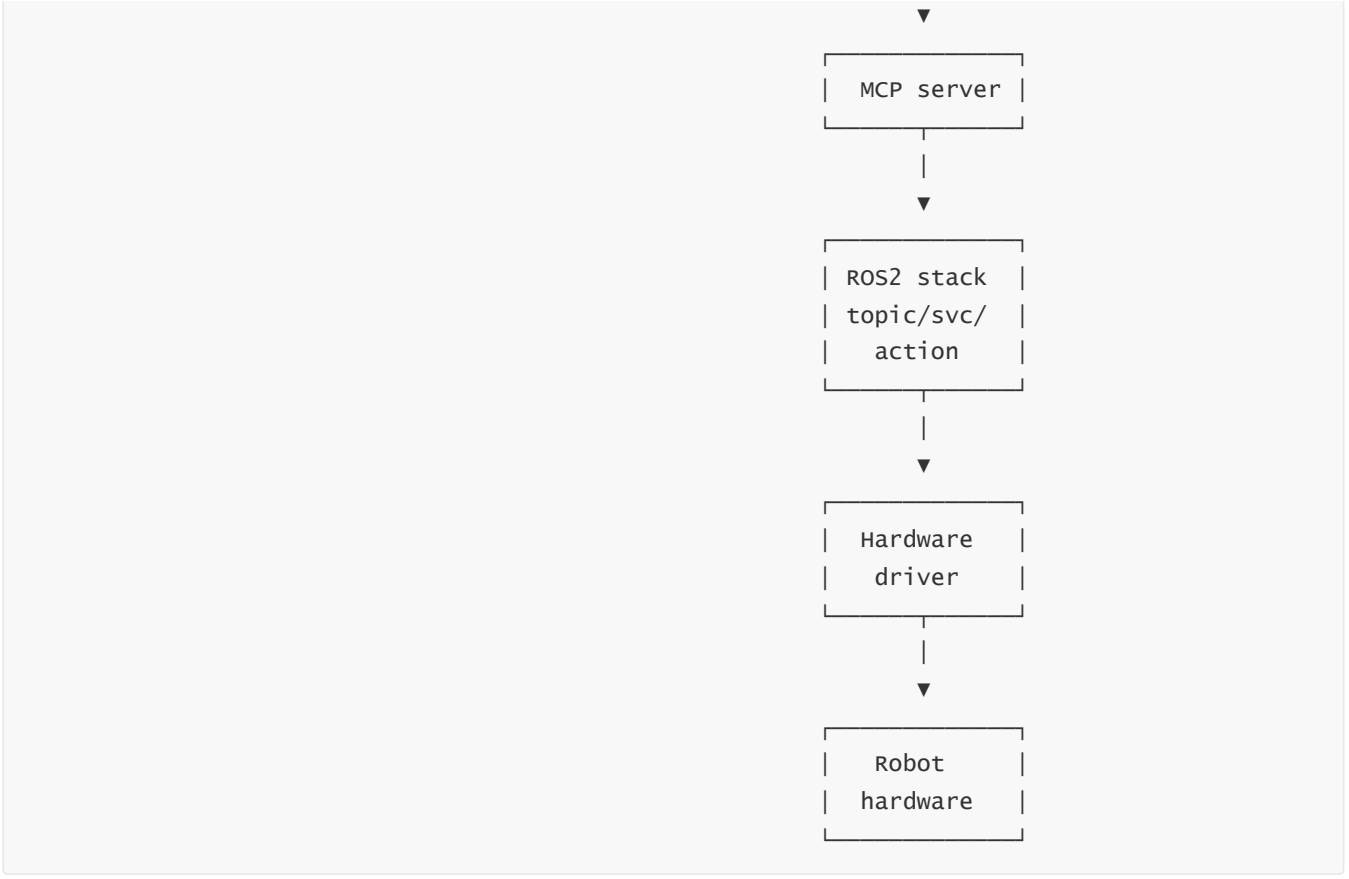
3. Introduction to the CLI Tool

The robot CLI (Command Line Interface) control tool is a command-line-based robot control interface. It calls the various robot control tools registered on the MCP server through the `robot_control` command. It encapsulates the robot's low-level control interfaces, converting complex ROS2 communication into concise command-line instructions. You only need to enter a command in the terminal to control the robot to perform various actions.



4. CLI Command Robot Control Communication Architecture

```
                                    ┌──────────────┐
                                    │   CLI tool   │
                                    └──────┬───────┘
                                           │MCP protocol
```



5. Controlling the Robot via CLI

5.1 CLI Command Summary

- Use `robot_control list-tools` to view the list of all available CLI commands.

```
robot_control list-tools
```

```
jetson@yahboom:~$ robot_control list-tools
  MoveWithSpeed(linear_x: number = ..., linear_y: number = ..., angular_z: number = ..., duration: number = ...)
    Control the robot's omnidirectional chassis movement using velocity commands

  Move(value: number, direction: string = ...)
    Control the robot's chassis to move a specified distance or rotate a specified angle

  GetCurrentArmJointAngles()
    Get the current joint angles of the robotic arm
```

- Below is a summary of all CLI commands:

Command	Parameter	Function	Camera Type
`list_tools`	None	List all tools currently available on the server	General (tool type)

Command	Parameter	Function	Camera Type
`MoveWithSpeed`	`--linear_x`, `--linear_y`, `--angular_z`, `--duration`	Control the omnidirectional chassis movement with velocity commands (linear velocity / angular velocity + duration)	General
`Move`	`--value`, `--direction`	Control the chassis to move a specified distance or rotate a specified angle. `value` is the distance (meters) or angle (degrees); `direction` can be: `forward`, `backward`, `left`, `right`, `turn_left`, `turn_right`	General
`GetCurrentArmJointAngles`	None	Get the current joint angles of the robotic arm	General
`ControlSixArmJoint`	`--arm_joint_1`~`--arm_joint_6`, `--runtime`	Control the 6 joints of the robotic arm simultaneously; the 6th joint is the gripper angle [0,180] (180=fully open, 0=fully closed)	General
`ControlSingleArmJoint`	`--arm_joint_id`, `--angle`, `--runtime`	Set a single joint of the robotic arm to a specified angle (id 1-6, the 6th is the gripper)	General
`AdjustCameraView`	`--pitch`, `--yaw`	Adjust the camera view and return the adjusted image. pitch positive=up/negative=down, yaw positive=right/negative=left; a step of 5-10 degrees is recommended	General
`SeeWhat`	None	Capture a frame of image and return the saved path	General
`InitArmPose`	None	Restore the robotic arm to the preset initial pose	General
`GetBbox`	`--query`	Detect and return the bounding box (bbox) coordinates of the target object based on a text description	General
`AdjustChassisFitArmRange`	`--x`, `--y`, `--z`	Quickly adjust the mobile chassis so that the target point enters the robotic arm's working range	General
`RecordMapLocation`	`--name`, `--symbol`	Save the current position to the map mapping file	General
`GetMapMapping`	None	Get the mapping between all place names and their corresponding symbols	General
`Navigation`	`--location`	Navigate to the target position via the position symbol in the map mapping file	General
`TTS`	`--text`	Robot text-to-speech broadcast	General

Command	Parameter	Function	Camera Type
`TargetTrack`	`--x1_or_cmd`, `--y1`, `--x2`, `--y2`	Target object visual tracking; pass `cancel` to `x1_or_cmd` to stop tracking, or pass an integer to use it as the bbox x1 to start tracking	General
`GraspArUcoTarget`	`--target_id`	Grasp the object marked with an AprilTag by recognizing its AprilTag ID	General
`GetAprilTagIDs`	None	Get the IDs of all AprilTags currently in view	General
`FollowLine`	`--color`	Drive along a colored line on the ground. `color` can be: `red`, `green`, `blue`, `yellow`	General
`Place`	`--place_x`, `--place_y`, `--place_z`	Place the object at the specified XYZ coordinates	Depth camera only
`Pick`	`--x1`, `--y1`, `--x2`, `--y2`	Grasp the object according to the bbox coordinates of the target object	Depth camera only
`GetPlacePoint`	`--query`	Calculate the placement point coordinates and determine whether it is within the robotic arm's reachable workspace	Depth camera only
`GetWasteRecognitionResults`	None	Get the waste recognition results of the YOLO model	Depth camera only
`GraspWaste`	`--waste_name`	Grasp the waste object with the specified name in the recognition results	Depth camera only
`PlaceWaste`	None	Place the waste object that has been grasped	Depth camera only
`GetTargetDist`	`--query`	Get the distance from the target to the robot based on the object description	Depth camera only
`GraspColor`	`--color`	Grasp a color block by color. `color` can be: `red`, `green`, `blue`, `yellow`	USB camera only
`PutDown`	None	Put down the currently grasped object: move to the put-down pose → open the gripper to release → return to the initial pose	USB camera only

5.2 Tools Supported by Different Camera Vehicle Models

- **Chassis movement**: `MoveWithSpeed`, `Move`, `AdjustChassisFitArmRange`

- **Robotic arm control**: `GetCurrentArmJointAngles`, `ControlSixArmJoint`, `ControlSingleArmJoint`, `InitArmPose`
- **Vision / camera**: `AdjustCameraView`, `SeeWhat`, `GetBbox`, `TargetTrack`
- **Navigation / map**: `RecordMapLocation`, `GetMapMapping`, `Navigation`, `FollowLine`
- **AprilTag**: `GraspArUcoTarget`, `GetAprilTagIDs`
- **Voice**: `TTS`
- **Depth camera only (7):** `Place`, `Pick`, `GetPlacePoint`, `GetWasteRecognitionResults`, `GraspWaste`, `PlaceWaste`, `GetTargetDist`
- **USB camera only (2):** `GraspColor`, `PutDown`

5.3 Command Format

All CLI commands follow a unified command format:

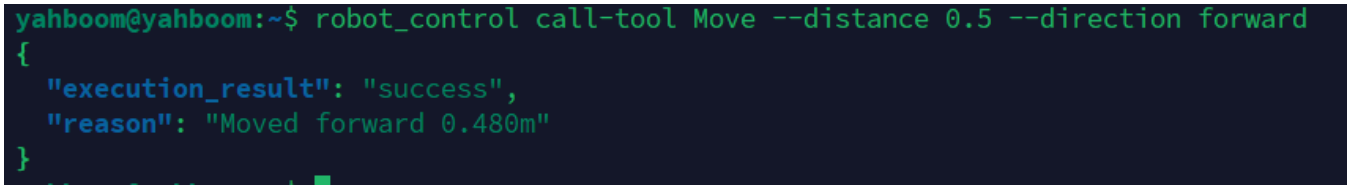
```
robot_control call-tool <command_name> --param1 <value1> --param2 <value2> ...
```

- `robot_control`: the main CLI tool command
- `call-tool`: indicates the tool invocation subcommand
- `<command_name>`: the specific tool name registered on the MCP server, such as `Move`, `SeeWhat`, `TTS`, etc.
- `--param <value>`: the parameter key-value pairs passed as needed; different commands have different numbers and types of parameters

6. Case Demonstration

- Move the robot chassis forward 0.5 meters

```
robot_control call-tool Move --value 0.5 --direction forward
```



- Capture the environment image

```
robot_control call-tool SeeWhat
```

```
jetson@yahboom:~$ robot_control call-tool SeeWhat
{
  "result": {
    "image": "/home/jetson/yahboomcar_ros2_ws/yahboomcar_ws/cache/image.png"
  }
}
```

> **Note**
>
> When the SeeWhat tool is called, the image is automatically saved to `$HOME/yahboomcar_ros2_ws/yahboomcar_ws/cache/image.png`

7. Source Code Analysis

Source code path:

```
~/yahboomcar_ros2_ws/yahboomcar_ws/src/robot_util/cli/robot_control
```

The source code of the CLI tool registers each robot control function as an MCP Tool through the `register_actions()` function in `mcp_action.py`. Here, the `Move` (move the chassis a specified distance) tool is used as an example for the source code explanation:

```
@call_tool_app.command(name='Move')
async def Move(
    *,
    direction: Annotated[str, cyclopts.Parameter(help="Movement direction: 'forward', 'backward', 'turn_left', or 'turn_right'")],
    distance: Annotated[float, cyclopts.Parameter(help="Linear movement distance in meters. Only used when direction is 'forward' or 'backward'. Set to 0.0 for rotation.")] = 0.0,
    angle: Annotated[float, cyclopts.Parameter(help="Rotation angle in degrees. Only used when direction is 'turn_left' or 'turn_right'. Set to 0.0 for linear movement.")] = 0.0,
) -> None:
    '''Move the robot chassis by a specified distance or rotate by a specified angle using odometry feedback.'''
    await _call_tool('Move', {'direction': direction, 'distance': distance, 'angle': angle})
```

Code explanation:

1. **@call_tool_app.command(name='Move')**: uses the `cyclopts` framework to register the function as a CLI subcommand; `name` defines the command name, corresponding to the tool name registered on the MCP server
2. **Function parameters**: CLI parameters are defined through `Annotated` type annotations + `cyclopts.Parameter`, and `help` sets the parameter description. `direction` is a required parameter, while `distance` and `angle` are optional parameters with a default value of 0.0
3. **_call_tool**: a unified tool invocation function that connects to the MCP server internally through `fastmcp.Client`, filters the parameters and sends them to the server for execution, then prints the returned result

Parameter description:

- `direction`: required parameter, movement direction, possible values: `forward`, `backward`, `turn_left`, `turn_right`
- `distance`: optional parameter, linear movement distance, unit: meters, only used for the `forward/backward` directions, default value 0.0
- `angle`: optional parameter, rotation angle, unit: degrees, only used for the `turn_left/turn_right` directions, default value 0.0
- `Annotated[type, cyclopts.Parameter(help)]`: the cyclopts type annotation, which adds help information for each CLI parameter so that users can view the parameter description by executing `--help` in the terminal
- `*`: forces subsequent parameters to be passed by name (i.e., the `--param value` format must be used)